STACKS AND QUEUES

Problem Solving with Computers-II



stack offers a **reduced set of operations efficiently (all O(1))**

A stack is useful for keeping track of history information where computation only depends on the most recent information !!

Stack has only three operations: **push()** **pop()** **top()**

stack<int> s

Empty stack



Operations: **push()** **pop()** **top()**

stack<int> s
s.push(70)



Operations: **push()** **pop()** **top()**

stack<int> s

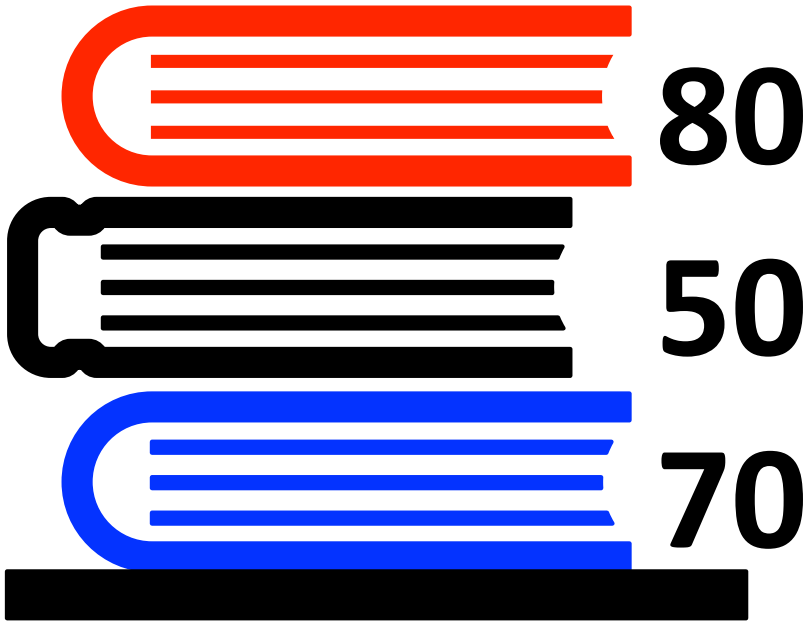
s.push(70)

s.push(50)



Operations: **push()** **pop()** **top()**

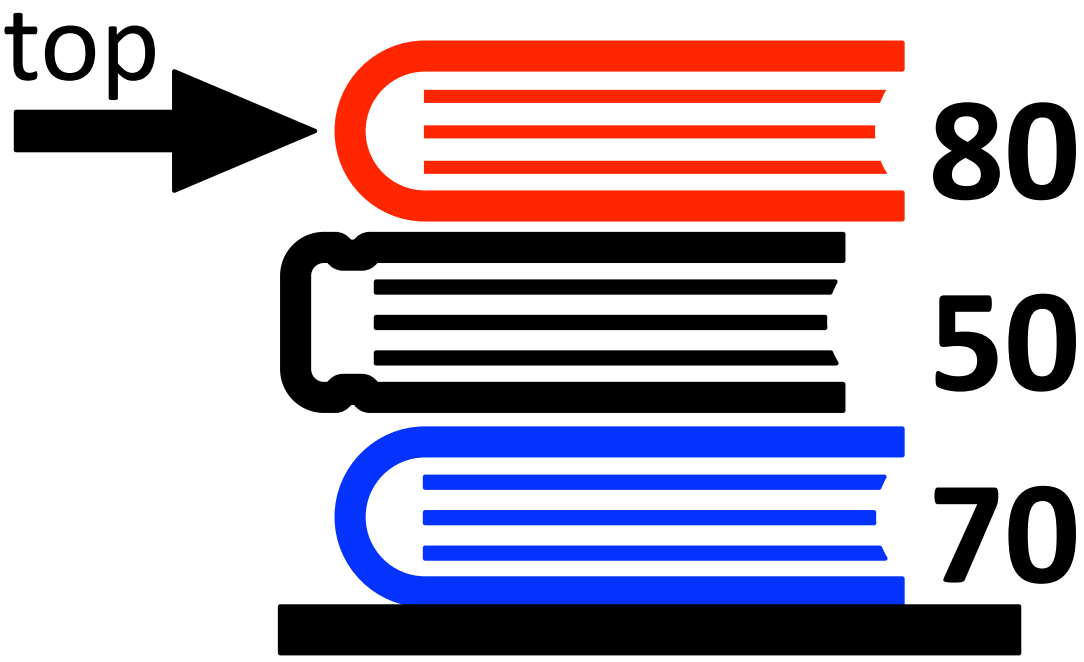
```
stack<int> s
s.push(70)
s.push(50)
s.push(80)
```



Operations: **push()** pop() top()

```
stack<int> s
s.push(70)
s.push(50)
s.push(80)
```

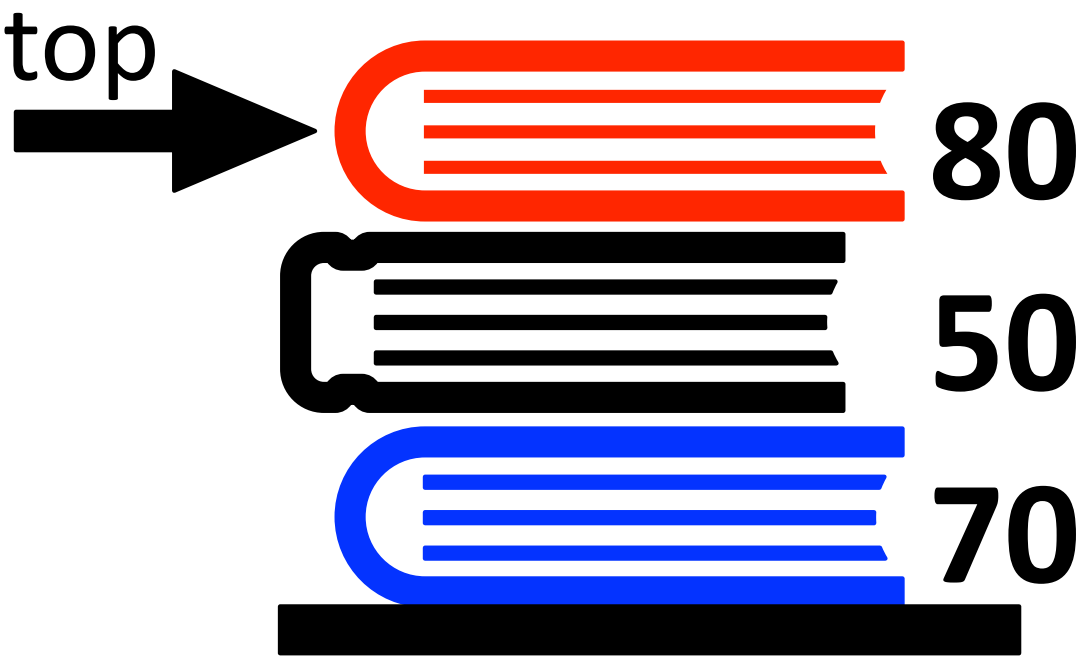
s.top() returns 80



Operations: push() pop() **top()**

```
stack<int> s
s.push(70)
s.push(50)
s.push(80)

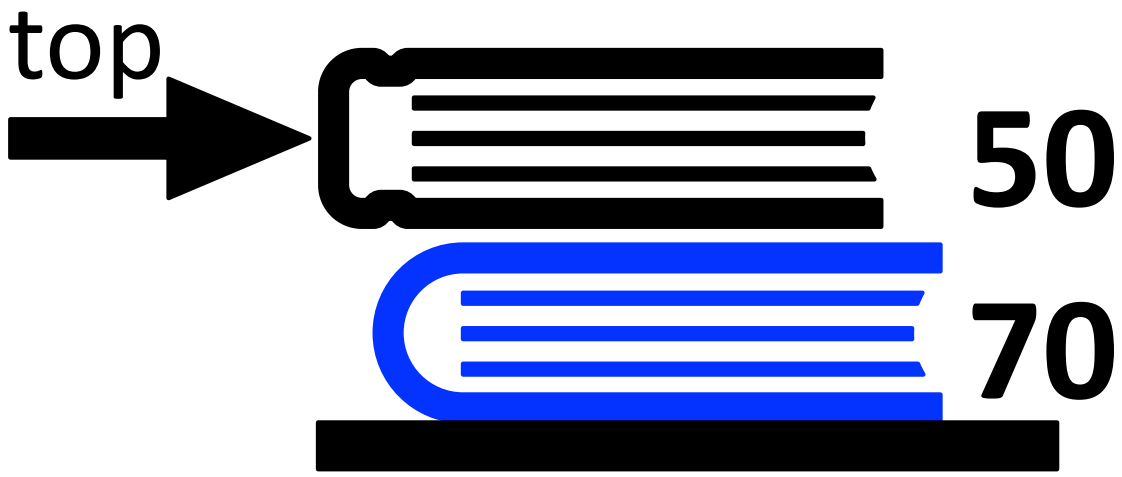
s.top()
```



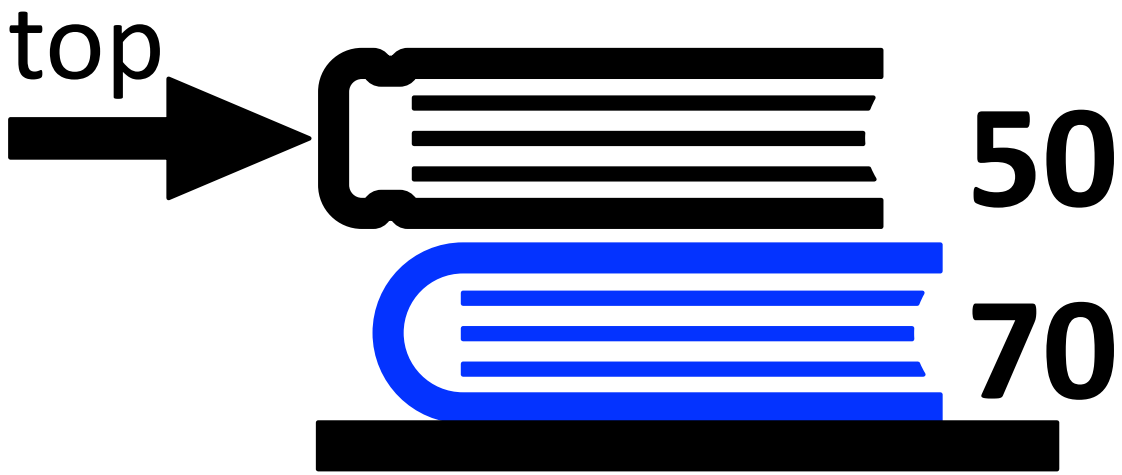
s.pop() removes value that was pushed in *last*

```
stack<int> s
s.push(70)
s.push(50)
s.push(80)

s.top()
```



s.pop() removes value that was pushed in *last*



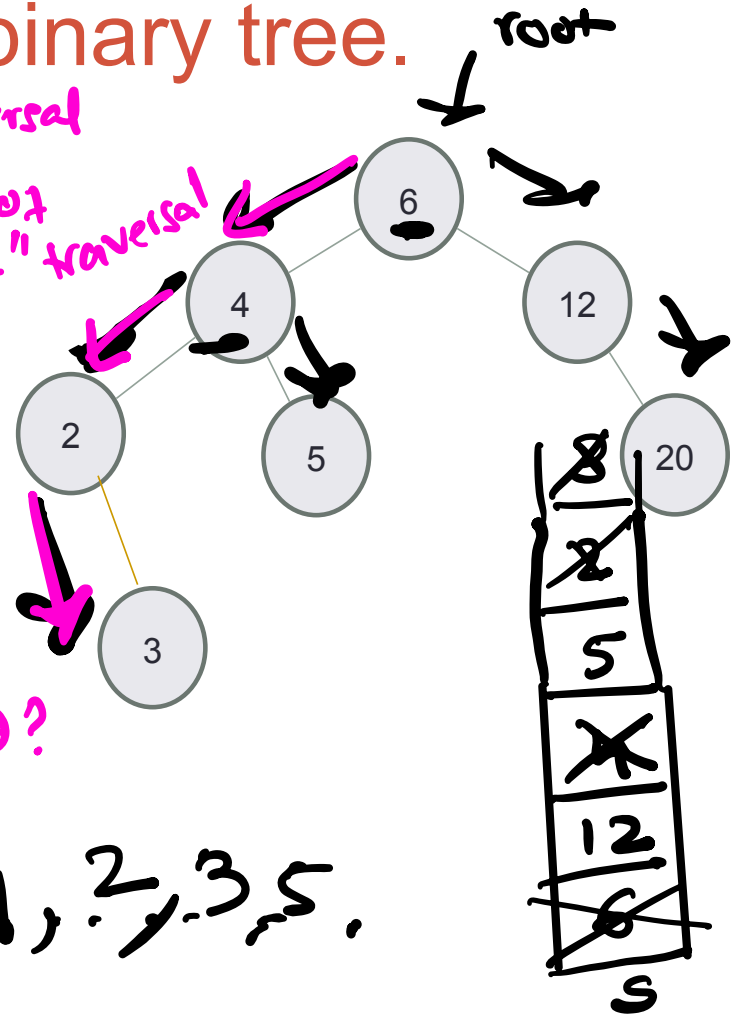
The Last value In is the First value Out (LIFO)

iclicker: Trace the code on the binary tree.

```
mystery(root):
    s = empty stack
    if root not null: s.push(root)
    while s not empty:
        n = s.top(); s.pop()
        print n.key
        if n.right not null: s.push(n.right)
        if n.left  not null: s.push(n.left)
```

Algo is an iterative preorder traversal
||
"an example of "Depth-First" traversal

"visit the node n"



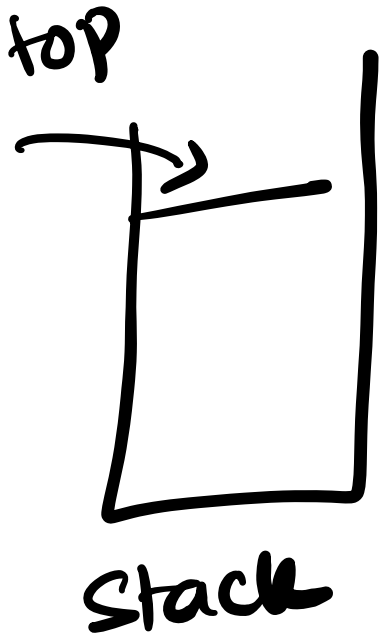
In what order are the keys in the binary tree printed? / visited?

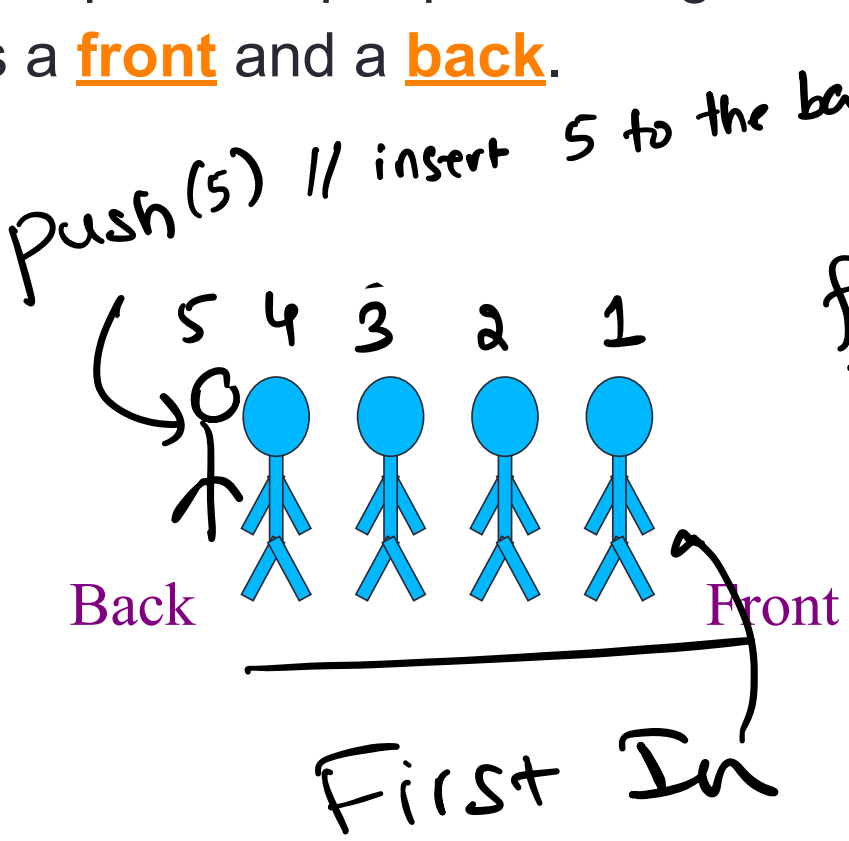
- A. Inorder [2, 3, 4, 5, 6, 12, 20]
- B. Postorder [3, 2, 5, 4, 20, 12, 6]
- C. Preorder [6, 4, 2, 3, 5, 12, 20]
- D. Something else

6, 4, 2, 3, 5.

Queue: First come First Serve

- A queue is like a queue of people waiting to be serviced
- The queue has a **front** and a **back**.



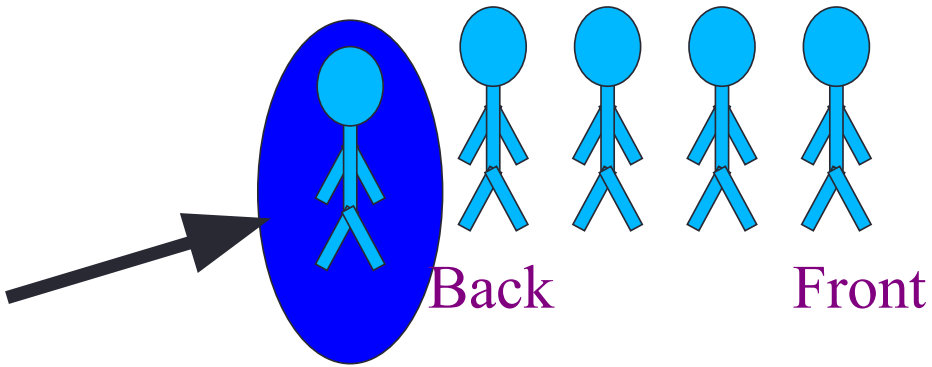


front() // 1
pop() // 1 is removed
2 is front

First Out.

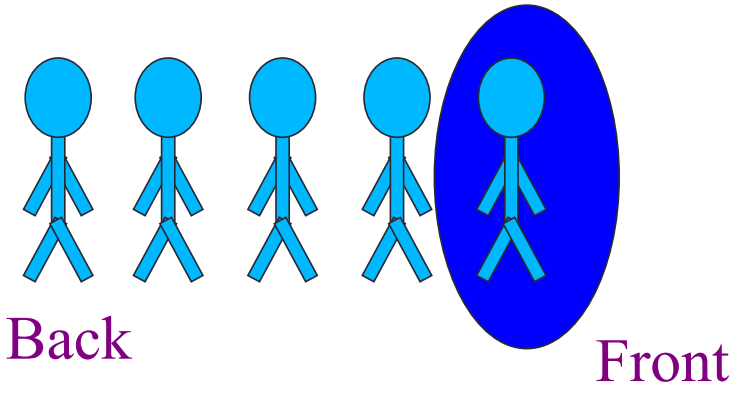
Queue Operations: push, pop, front, back

New people must enter the queue at the back. The C++ queue class calls this a **push** operation.



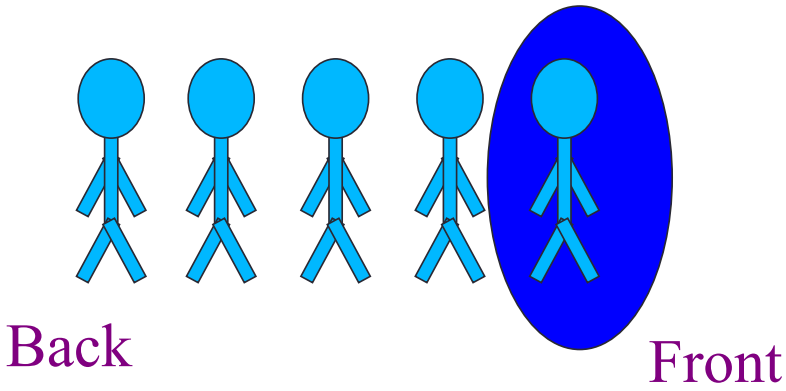
Queue Operations: push, pop, front, back

- To check the item in the front of the queue, use **front()**
- To check the item at the back of the queue, use **back()**
- When an item is taken from the queue, it always comes from the front.
- To delete an element from the front of the queue, use **pop()**



Queue Operations: empty(), push, pop, front, back: **O(1)**

```
std::queue<int> q;
q.empty(); //true
q.push(1);
// push 2, 3, 4, 5
q.front();
q.back();
q.pop();
```



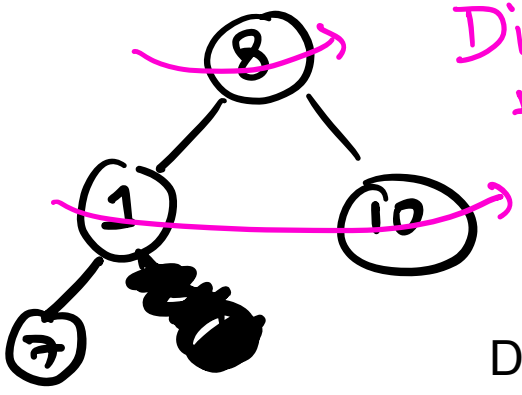
Game: Explore the secret tree (pairs, 8 min)

Mapper

- Draw a secret binary tree: 7 distinct keys, about 3 levels.
- Tell the Explorer **only the root**.
- Answer only "What are the children of X?" with left, right or none.

Explorer

- Start: q.push(root)
- Repeat until the queue is empty:
  1. x = q.front(); q.pop();
  2. Write x in the visit order
  3. Ask the Mapper for the children of x
  4. q.push(child), left first, then right

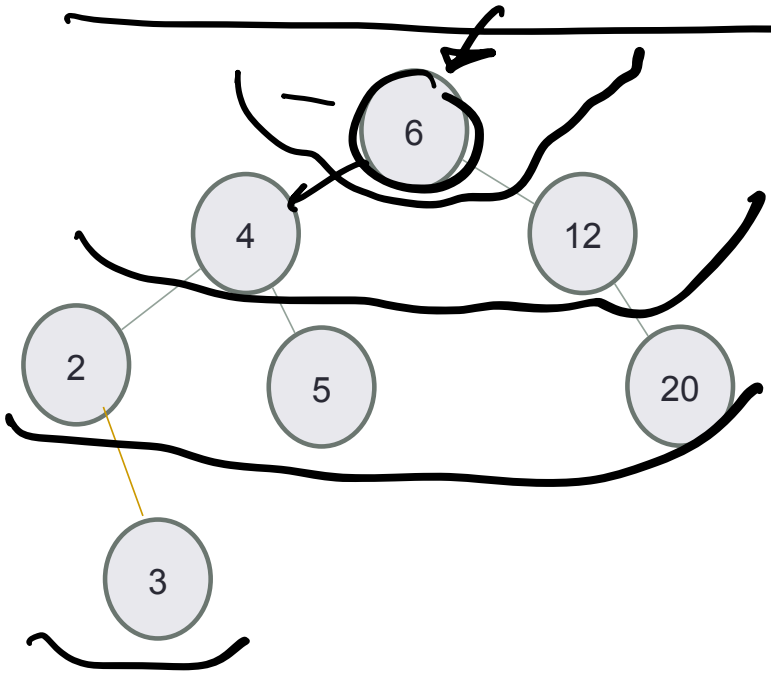


Discover the traversal pattern! Visit order: 8, 1, 10, 7...

Step	Queue before this step (Back ... Front)	front() popped	Children pushed
1	8	8	1, 10
2	10, 1	1	7, null
	7, 10		

Done early? Swap roles with a new tree.

Breadth-first traversal/search



BFS Algo:

- Create an empty **queue**.
- Insert the **root** into the **queue**.
- While queue is not empty,
  - Print the key in the front of the queue
  - Pop the front node from the queue
  - Push all the children of the node into the queue.

BFS visit order

6, 4, 12, 2, 5, 20, 3

live code

ADT	Algorithm	Real-World Application
Stack	Depth-First Search	Machine Learning (**PA03:** Training and Prediction in Neural Nets)
Queue	Breadth-First Search	
Priority Queue	Dijkstra's Algorithm	GPS Navigation (Shortest path)
	Huffman Coding	Data Compression (ZIP, JPEG, MP3)
Your choice!	You design!	Querying a movie dataset **(PA02)**

